Grid Counting

This problem seems to tell you to

1. Count all sudokus with a 3 in the corner.
2. Count all sudokus.
3. Determine the value of the fraction.

Instead, there is a much more simple way of solving the problem.

1. Realize that the fraction can be determined without counting.
2. Immediately get a partial score for each subproblem.
3. Only count all sudokus, with no constraints on their content.
4. From the fraction and the total count, determine the count of sudokus with a 3 in the corner.

Determining the fraction

Obviously, having a 3 in two corners that share a side violates already the rules for a Latin square and thus this cannot be a valid sudoku either. Hence, there are only sudokus with exactly one 3 in a corner, and sudokus with two 3s in diagonally opposite corners.

By symmetry, the number x of sudokus with a 3 in the top left corner is the same as for any other corner, and the number y for two 3s on the main diagonal is the same as the number of sudokus where the two 3s are on the other diagonal. This means that by a simple inclusion-exclusion argument, the total number of sudokus with a 3 in a corner is $4x - 2y$.

The obvious part is that $x = t/n$, where t is the total number of all sudokus.

One possible proof: Call two sudokus equivalent if one can be obtained from the other by adding, modulo n, the same value to each cell. Each equivalence class then has n sudokus that differ in the top left corner, and thus exactly one in n sudokus has a 3 in the top left corner.

The less obvious part is that $y = t/n^2$. Intuitively, it might seem that putting a 3 into the top left corner of a sudoku grid should somehow influence the probabilities of what might appear in the bottom right corner, but it does not.

In fact, a more general statement is true:

Let's call a group of rows that intersect the same set of boxes a *band*, and a group of columns going through the same boxes a *stack*.

The claim is that whenever we pick two cells that are both in different bands and in different stacks, the two cells are independent — for any two values (p, q) the probability that the first cell is p and the second is q if we pick a valid random sudoku is exactly $1/n^2$. Or, in other words, the total number of such sudokus is exactly t/n^2.

One possible proof: Call two sudokus equivalent if one can be transformed to the other by doing the following:

1. Cyclically rotate the rows of the band that contains our first cell.
2. Cyclically rotate the columns of the stack that contains our first cell.
3. Cyclically rotate the rows of the band that contains our second cell.
4. Cyclically rotate the columns of the stack that contains our second cell.

Clearly, there are exactly n ways how to perform steps 1+2 and each of them puts a different value into our first cell. And once that is done, there are exactly n ways how to do steps 3+4 and each of them puts a different value into our second cell. Thus, each pair (p, q) of values in our two cells appears in exactly $1/n^2$ of all sudokus.

Hence, the correct fraction is always $(4n - 2)/n^2$. (This is not necessarily its reduced form.)

Counting empty sudokus

The value $t_4 = 288$ was explicitly given.

The value $t_6 = 28\,200\,960$ can be obtained by simple brute force. Or by even simpler and faster brute force once we realize that we can fill the first row with 123456 and then multiply the final count by $6!$.

We will explain in detail one way to compute $t_8 = 29\,136\,487\,207\,403\,520$ within a few minutes.

We can again fix the first row to 12345678 and multiply by $8!$ at the end. The second row of the first band must then contain the digits 5 to 8 in the left stack and 1 to 4 in the right stack, so there are $4! \cdot 4! = 576$ possible first bands.

We don't need to process each of these 576 first bands separately. Instead, at this point we can look for additional symmetries.

Here's one: we can arbitrarily permute the columns of the left stack and independently of that permute the columns of the right stack. Once we do that, we can relabel all values so that the top row reads 12345678 again. This way, each specific pair of such column permutations defines one bijection on the set of all sudokus with top row 12345678. We can say that two ways of filling the top band are *equivalent* if there is such a bijection transforming one of them into the other. Clearly, equivalent ways of filling the top band must lead to the same number of valid sudokus.

In this way, we can divide the 576 ways of filling the top band into just five orbits (equivalence classes). Their sizes are 24, 144, 72, 192, and 144, respectively.

For each orbit separately we now pick one representative, count the number of ways to fill the rest of the grid, and then multiply that by the size of the orbit. This counting can now be done in a quite straightforward way. We'll proceed one band at a time, and memoize the number of ways to finish the grid on the sets of digits already used in each column of the grid. The last band is cheap: each column has exactly two digits left, we only need to decide which of them goes on bottom so that the bottommost row contains every digit once.

(For each of the five representatives we'll get about 10^9 valid ways to fill in the rest of the grid. These five counts are all distinct, so with this approach we did actually need to process each orbit separately.)

Efficient approaches to the computation of $t_9 = 6\,670\,903\,752\,021\,072\,936\,960$ follow pretty much in the same footsteps. For more details on what additional optimizations are enough to compute the answer within contest time, a good starting point is the historically first approach used to compute this value: the Felgenhauer–Jarvis algorithm.